

```
In [1]: # =====
# Notebook setup
# =====

%load_ext autoreload
%autoreload 2

# Retrain or use existing models
use_cached_weights = True

# Control figure size
figsize=(14, 3.5)
figsize_narrow=(14, 2.5)

from util import util
import warnings
warnings.filterwarnings("ignore", ".*Consider increasing the value of the `r
warnings.filterwarnings("ignore", ".*GPU available but not used.*")
warnings.filterwarnings("ignore", ".*Previous log files in this directory wi
warnings.filterwarnings("ignore", ".*Checkpoint directory.*exists and is not

# Generate a problem instance
nitems, rel_req, seed = 20, 0.5, 42
prb = util.generate_problem(nitems=nitems, rel_req=rel_req, seed=seed)

# Generate basic dataset
data_tr = util.generate_costs(nsamples=350, nitems=nitems, seed=seed, noise_
data_ts = util.generate_costs(nsamples=150, nitems=nitems, seed=seed, sampli
```

Applicability of DFL

Let's Second-Guess Ourselves

However, let's not discount the prediction-focused approach yet

In fact, it's easy to see that:

$$\mathbb{E}[\text{regret}(y, \hat{y})] \xrightarrow{\mathbb{E}[L(y, \hat{y})] \rightarrow 0} 0$$

Intuitively:

- The more accurate we can be, the lower the regret
- Eventually, perfect predictions will result in 0 regret

Overall:

- If our model is expressive enough (i.e. the model is *not misspecified*)

- ...And the problem is *deterministic*

Then the benefits of DFL will be *vanishingly small*

Let's Second-Guess Ourselves

But it gets worse! Let's check again our decision problem:

$$z^*(\hat{y}) = \operatorname{argmin}_z \left\{ \hat{y}^T z \mid z \in F \right\}$$

Under this assumption we have that:

$$\begin{aligned} & \operatorname{E}_{x,y \sim P(X, Y)} [\text{regret}(y, \hat{y})] \\ &= \operatorname{E}_{x,y \sim P(X, Y)} [y^T (z^*(\hat{y}) - z^*(y))] \\ &= \operatorname{E}_x [\operatorname{E}_{y \sim P(Y \mid x)} [y^T] (z^*(\hat{y}) - z^*(y))] \end{aligned}$$

- ...Which means that, if we can accurately predict the y expectation
- ...Then we will be able to make provably optimal decisions

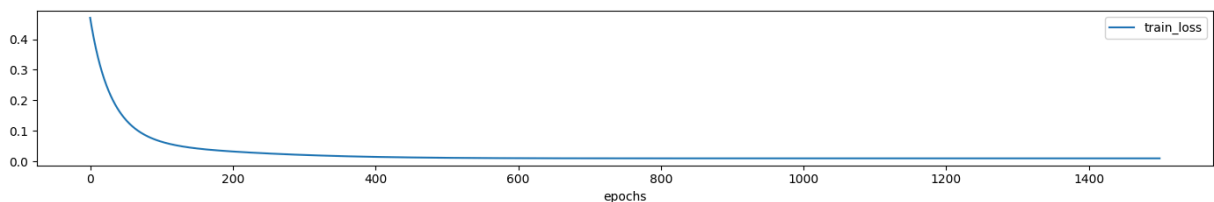
As long as only *the coefficient of a linear cost* are uncertain

...Then the DFL advantage will also be vanishingly small!

Our Baseline

Let's check again the results for our PFL linear regressor

```
In [2]: pfl = util.TwoPhaseModel(input_size=1, output_size=nitems, hidden_sizes=[],
pfl, history, metadata = util.train_ml_model(pfl, data_tr.index.values.reshape(-1, 1),
util.plot_training_history(history, metadata=metadata, figsize=figsize_narrow,
util.print_ml_metrics(pfl, data_tr.index.values.reshape(-1, 1), data_tr.values.reshape(-1, 1),
util.print_ml_metrics(pfl, data_ts.index.values.reshape(-1, 1), data_ts.values.reshape(-1, 1), data_ts.values.reshape(-1, 1))
```



training time: 9.5323

R2: 0.829, MAE: 0.081, RMSE: 0.099 (training)

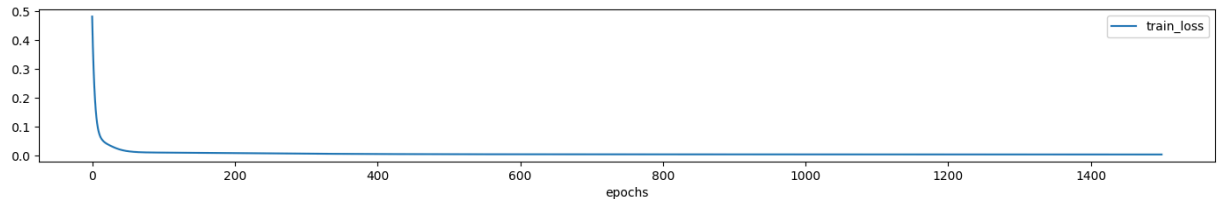
R2: 0.823, MAE: 0.083, RMSE: 0.101 (test)

This will be our main baseline

PFL Strikes Back

Let's try to use a *non-linear model*

```
In [3]: pfl_nl = util.TwoPhaseModel(input_size=1, output_size=nitems, hidden_sizes=[16],
pfl_nl, history, metadata = util.train_ml_model(pfl_nl, data_tr.index.values.reshape(-1, 1), data_tr.values.reshape(-1, 1))
util.plot_training_history(history, metadata=metadata, figsize=figsize_narrow)
util.print_ml_metrics(pfl_nl, data_tr.index.values.reshape(-1, 1), data_tr.values.reshape(-1, 1))
util.print_ml_metrics(pfl_nl, data_ts.index.values.reshape(-1, 1), data_ts.values.reshape(-1, 1))
```



training time: 10.8216

R2: 0.953, MAE: 0.040, RMSE: 0.053 (training)

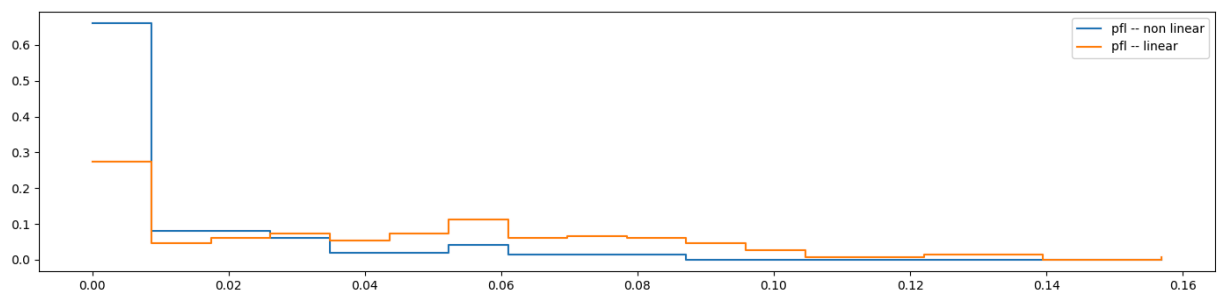
R2: 0.952, MAE: 0.040, RMSE: 0.054 (test)

More accurate, it is!

PFL Strikes Back

...And the improvement in terms of regret is remarkable

```
In [4]: r_ts = util.compute_regret(prb, pfl, data_ts.index.values.reshape(-1, 1), data_ts.values.reshape(-1, 1))
r_ts_nl = util.compute_regret(prb, pfl_nl, data_ts.index.values.reshape(-1, 1), data_ts.values.reshape(-1, 1))
util.plot_histogram(r_ts_nl, figsize=figsize, label='pfl -- non linear', data=r_ts_nl)
```

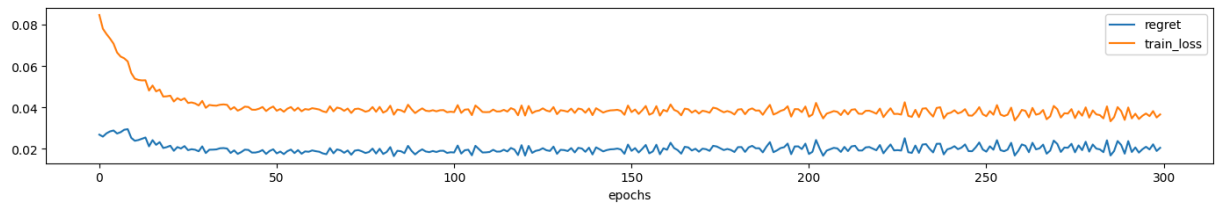


Mean: 0.012 (pfl -- non linear), 0.043 (pfl -- linear)

PFL Strikes Back

Let's what DFL could do with the same model complexity

```
In [5]: spo_nl = util.DFLModel(input_size=1, output_size=nitems, hidden_sizes=[16],
spo_nl, history, metadata = util.train_ml_model(spo_nl, data_tr.index.values.reshape(-1, 1), data_tr.values.reshape(-1, 1))
util.plot_training_history(history, metadata=metadata, figsize=figsize_narrow)
util.print_ml_metrics(spo_nl, data_tr.index.values.reshape(-1, 1), data_tr.values.reshape(-1, 1))
util.print_ml_metrics(spo_nl, data_ts.index.values.reshape(-1, 1), data_ts.values.reshape(-1, 1))
```



training time: 79.5560

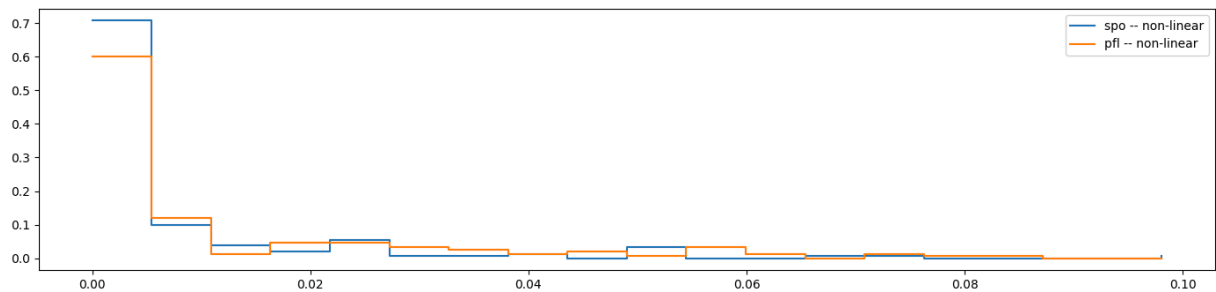
R2: 0.874, MAE: 0.067, RMSE: 0.085 (training)

R2: 0.870, MAE: 0.068, RMSE: 0.086 (test)

PFL Strikes Back

The gap between the approach is closed, or even inverted

```
In [6]: r_ts_spo_nl = util.compute_regret(prb, spo_nl, data_ts.index.values.reshape(
fig = util.plot_histogram(r_ts_spo_nl, figsize=figsize, label='spo -- non-li
fig.savefig('pfl_dfl_shallow.pdf')
```



Mean: 0.008 (spo -- non-linear), 0.012 (pfl -- non-linear)

Reflecting on What we Have

It's worth now reassessing our current situation

When we have just a *linear cost* with *uncertain coefficients*:

- If the predictive model is *misaligned*, DFL can provide a benefit
- Otherwise, PFL is faster and leads to the same (or better) decisions

This is still useful!

- We can use DFL to *do more with simpler models*
- ...And get better explainability, easier analysis, etc.

...But if we wish for consistent benefits, we need to *relax our assumptions*

- For example, we can allow for *non-linear cost functions*
- ...And we are about to see one very common case when they arise

Two-Stage Stochastic Optimization

We will focus on two-stage stochastic optimization problems

Let's consider the problem of packing up for a trip



- Before the trip (*first stage*) we decide what to put in our suitcase
- During the trip (*second stage*), we may realize we have forgotten something
- ...And we need to *spend money* (*recourse action*) to buy the missing stuff

Two-Stage Stochastic Optimization

Two-stage stochastic optimization problems (2s-SOP) are defined as:

$$\operatorname{argmin}_z \left\{ f(z) + \mathbb{E}_{y \sim P(Y|x)} \left[\min_{z''} r(z'', z, y) \right] \mid z \in F, z'' \in F''(z, y) \right\}$$

- Y represents the uncertain information
- z is the vector of *first stage decisions*
- F is the feasible space for the first stage
- z'' is the vector of *recourse actions*
- z'' is not fixed: it can change for every sampled y
- The set of feasible recourse actions $F''(z, y)$ also changes for every y
- f is the immediate cost function, r is the cost of the recourse actions

A Simple Example

We will consider this simple problem, based on our supply planning example:

$$\operatorname{argmin}_z c^T z + \mathbb{E}_{y \sim P(Y|x)} \left[\min_{z''} c'' z'' \right] \quad (2)$$

$$\text{subject to: } y^T z + z'' \geq y_{\min} \quad (3)$$

$$z \in \{0, 1\}^n, z'' \in \mathbb{N}_0 \quad (4)$$

- $z_j = 1$ iff we choose then j -th supply contract
- c_j is the cost of the j -th contract
- y_j is the yield of the j -th contract, which is uncertain

- y_{min} is the minimum total yield, which is known
- z'' is the number of units we buy at cost c'' to satisfy the yield requirement

Non-linearities in the cost arise here due to the recourse actions

Scenario Based Approach

Classical solution approaches for 2s-SOP are *scenario based*

We start by estimating $P(Y | x)$ and sampling a set of N values:

$$\operatorname{argmin}_z \min_{z''} c^T z + \frac{1}{N} c'' z''_k \quad (5)$$

$$\text{subject to: } y^T z + z''_k \geq y_{min} \quad \forall k = 1..N \quad (6)$$

$$z \in \{0, 1\}^n \quad (7)$$

$$z''_k \in \mathbb{N}_0 \quad \forall k = 1..N \quad (8)$$

Then we build different recourse action variables for each scenario

- ...We define the feasible sets via constraints
- ...And we use the Sample Average Approximation to estimate the expectation

The method is effective, but also *very computationally expensive*

...And Deterministic Approximations

Hence, *deterministic approximations* are very common in practice

Typically, one optimizes for the *most likely* scenario:

$$z^*(y) = \operatorname{argmin}_z \min_{z''} c^T z + c'' z''_k \quad (9)$$

$$\text{subject to: } y^T z + z''_k \geq y_{min} \quad (10)$$

$$z \in \{0, 1\}^n \quad (11)$$

$$z''_k \in \mathbb{N}_0 \quad (12)$$

By doing this:

- The scalability is comparable with 1-stage problems
- We can obtain the scenario via a classic predictor (point estimator)
- ...But we no longer properly account for uncertainty

DFL for 2s-SOP

We can use DFL principles to address problems in this class

Intuitively, the approach works as follows

- We observe x and we compute $\hat{y}$
- We compute $z^*(\hat{y})$ by solving a single-scenario problem
- We compute $\text{regret}(y, \hat{y})$ by solving a single-scenario problem with fixed z

By doing this:

...We end up solving a problem very close to 2s-SOP

- We will have a couple of limitations
- ...But also several major advantages

DFL for 2s-SOP

To get there, we'll need to adjust a bit our formulation

In particular, we'll consider a *single-scenario 2s-SOP*:

$$z^*(y) = \operatorname{argmin}_z \{f(z) + r(z'', z, y) \mid z \in F, z'' \in F''(z, y)\}$$

...And the *post-hoc* cost evaluation function:

$$g(z, y) = \min_{z''} \{f(z) + f(z'', z, y) \mid z'' \in F''(z, y)\}$$

I.e. the cost of the first-stage decision z under scenario y

- The first problem is a more general version of those used in classic DFL
- The second function is new

DFL for 2s-SOP

Then we can formulate this generalized DFL training problem

$$\operatorname{argmin}_\theta \mathbb{E}_{x, y \sim P(X, Y)} [g(z^*(\hat{y}, y)) - g(z, y)]$$

- where $g(z^*(\hat{y}, y)) - g(z, y)$ is called *post-hoc regret*
- ...or *value of perfect information*

For a given input x , this reduces to:

$$\operatorname{argmin}_\theta \mathbb{E}_{y \sim P(Y|x)} [g(z^*(\hat{y}, y)) - g(z, y)] = \tag{13}$$

$$\operatorname{argmin}_\theta \mathbb{E}_{y \sim P(Y|x)} [g(z^*(\hat{y}, y))] \tag{14}$$

where the right-end terms is very close to the classic 2s-SOP objective

Overview and Properties

Overall:

We only solve *single-scenario problems*

- This is true both at training and inference time
- ...And leads to massively better scalability than scenario-based methods

We can use a point estimator

- There's no need to estimate $P(Y \mid x)$
- ...Which might entail guessing the correct distribution type

However:

- Sometimes, using a single scenario **is not enough**
- ...And we are generating the scenario indirectly, by adjusting θ